

Problem Set 5

CMSC 27200: Theory of Algorithms

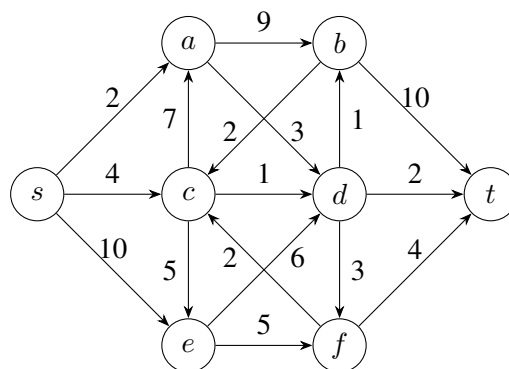
Assigned February 18, 2026, Due February 24, 2026

Problem 0: Collaborators and Outside Sources

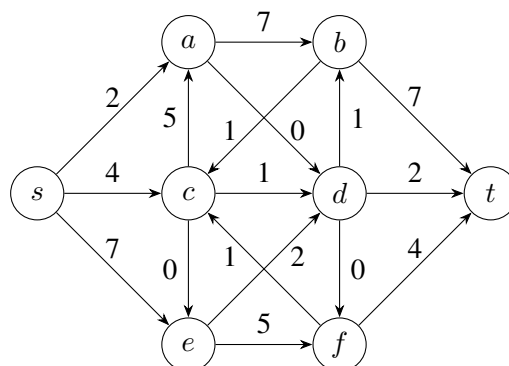
Who did you collaborate with? Did you use any outside sources?

Problem 1: Ford-Fulkerson (25 points)

Graph:



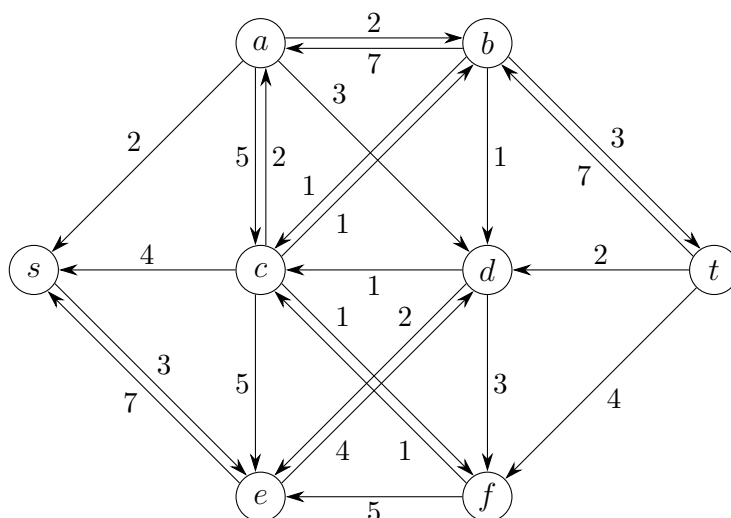
Current flow:



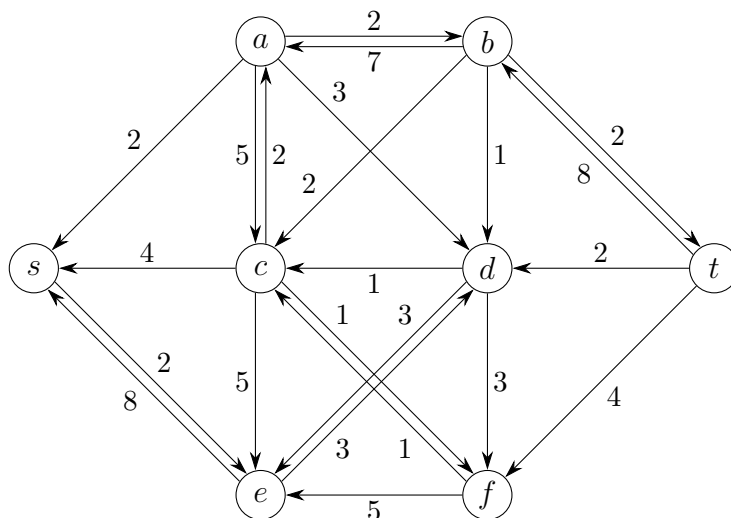
Starting from the given flow, implement the Ford-Fulkerson algorithm by hand to find a maximum flow from s to t and find a minimum cut proving that this flow is a maximum flow. For each iteration (including the iteration where the algorithm terminates), show the residual graph and the flow path that you find from s to t (if such a path exists).

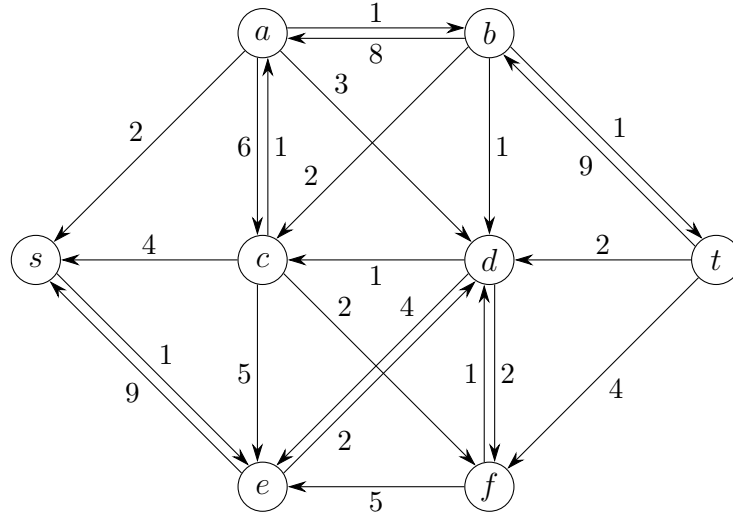
Solution

Residual graph of the given flow:

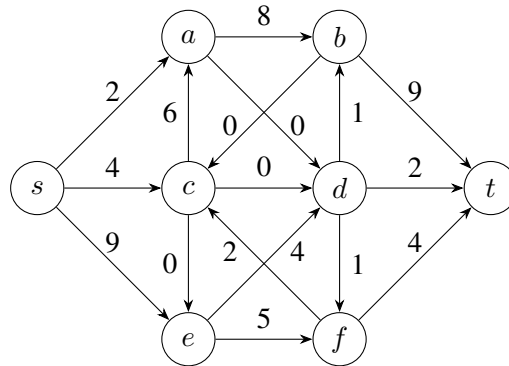


There is an s - t path $s \rightarrow e \rightarrow d \rightarrow c \rightarrow b \rightarrow t$ that we can use to push one unit of flow. The new residual graph will be the following:





Now there is no more s - t path in the residual graph and the following flow is a max-flow, the value of this flow is 15:



To find a min-cut with capacity equal to the value of this flow, let S be the set of vertices that are reachable from s in the final residual graph: $S = \{s, d, e, f\}$. Let $T = V \setminus S$. We claim that the capacity of (S, T) is equal to 15. Indeed, the edges pointing from S to T include (s, a) , (s, c) , (d, b) , (d, t) , (f, c) , (f, t) . The sum of the capacities is $2 + 4 + 1 + 2 + 2 + 4 = 15$.

Problem 2: Ford-Fulkerson With No Backwards Edges (20 points)

What happens if we don't have backwards edges in the Ford-Fulkerson algorithm? In other words, what happens if, after an iteration of Ford-Fulkerson where we find a path from s to t and route c_{min} flow through it (where c_{min} is the minimum remaining capacity of an edge on this path), we just reduce the capacities of the edges along this route by c_e in the residual graph (and don't increase the capacities of the edges going backwards by c_{min} as we are ignoring them)?

Your friend conjectures that the Ford-Fulkerson algorithm with no backwards edges will still always give a constant factor approximation, i.e., there exists a constant $c > 0$ such that if the maximum flow is F , regardless of how we choose the paths from s to t we will obtain a total flow of at least cF from s to t . Is your friend's conjecture correct? Either prove your friend's conjecture

or describe how to construct a counterexample for any $c > 0$.

Hint is available.

Solution

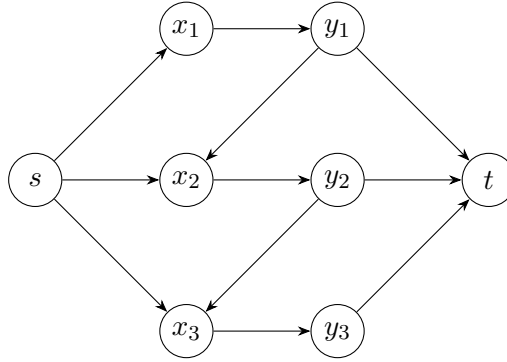
The conjecture is **false**. For any $c > 0$, a counterexample can be constructed as follows. Let $m = \lceil 1/c \rceil + 1$, and let $G = (V, E)$ where

$$V = \{s, t, x_1, \dots, x_m, y_1, \dots, y_m\}$$

and

$$E = \{(s, x_i) : 1 \leq i \leq m\} \cup \{(x_i, y_i) : 1 \leq i \leq m\} \cup \{(y_i, t) : 1 \leq i \leq m\} \cup \{(y_i, x_{i+1}) : 1 \leq i \leq m-1\}.$$

Each edge in E has capacity 1. The following is an example when $m = 3$:



The maximum flow of this graph is m (using m vertex-disjoint paths $s \rightarrow x_i \rightarrow y_i \rightarrow t$). However, if Ford-Fulkerson without backwards edges takes the path $s \rightarrow x_1 \rightarrow y_1 \rightarrow x_2 \rightarrow y_2 \rightarrow \dots \rightarrow x_m \rightarrow y_m \rightarrow t$ first, this saturates all edges (x_i, y_i) , leaving no further augmenting paths. This gives a total flow of only 1.

Since $m > 1/c$, we have $1/m < c$, so the ratio of the flow obtained to the maximum flow is $1/m < c$. Since $c > 0$ was arbitrary, the conjecture is false for any constant $c > 0$.

Problem 3: Store Supplier (25 points)

You are a supplier which supplies m different products to n stores. You have the following constraints:

1. For each product $i \in [m]$, you have a supply s_i of product i .
2. For each product $i \in [m]$ and each store $j \in [n]$, store j wants a maximum of c_{ij} of product i .
3. Each store $j \in [n]$ will buy a maximum of b_j products in total.

What is the maximum total number of products T that you can sell to these stores and which products should you sell to each store in order to achieve this? In other words, can you find $\{x_{ij} : i \in [m], j \in [n]\}$ such that

1. For all $i \in [m]$, $\sum_{j=1}^n x_{ij} \leq s_i$.
2. For all $i \in [m]$ and $j \in [n]$, $x_{ij} \leq c_{ij}$.
3. For all $j \in [n]$, $\sum_{i=1}^m x_{ij} \leq b_j$.

and your total sales $T = \sum_{i=1}^m \sum_{j=1}^n x_{ij}$ is maximized?

Letting $S = \sum_{i=1}^m s_i$, give an algorithm for this problem whose runtime is polynomial in S , m , and n . Prove that your algorithm is correct and analyze its runtime.

Hint is available.

Solution

Construct the following network G :

- Create a source vertex s and a sink vertex t . For each $i \in [m]$, create a vertex p_i and add an edge (s, p_i) with capacity s_i . For each $j \in [n]$, create a vertex q_j and an edge (q_j, t) with capacity b_j .
- For each $i \in [m]$ and each $j \in [n]$, add an edge (p_i, q_j) with capacity c_{ij} .

Then we run Ford-Fulkerson on G . Let F be the resulting max-flow. Let $x_{ij} = f(p_i, q_j)$ for all $i \in [m]$ and $j \in [n]$. Return $\{x_{ij}\}$.

Correctness: We show that there is an assignment $\{x_{ij}\}$ that achieves a total sales of T if and only if there exists a flow f with value T and $f(p_i, q_j) = x_{ij}$ for all $i \in [m]$, $j \in [n]$.

($\Rightarrow$) Let $\{x_{ij}\}$ be a solution to the problem that achieves total sales T . We define a flow with value T in G as follows. For all $i \in [m]$ and $j \in [n]$, let $f(p_i, q_j) = x_{ij}$, $f(s, p_i) = \sum_{j=1}^n x_{ij}$, and $f(q_j, t) = \sum_{i=1}^m x_{ij}$. Conservation of flow is obvious by construction. Constraints 1 to 3 imply the flows on edges (s, p_i) , (p_i, q_j) , and (q_j, t) respectively do not exceed the capacities. Thus f is indeed a flow. The value of f is precisely $\sum_{i=1}^m f(s, p_i) = \sum_{i=1}^m \sum_{j=1}^n x_{ij} = T$.

($\Leftarrow$) Let f be a flow on network G . If $v(f) = T$, we claim that $\{f(p_i, q_j)\}$ is a solution that achieves total sales T . Since $\sum_{i=1}^m \sum_{j=1}^n f(p_i, q_j) = \sum_{i=1}^m f(s, p_i) = T$, it remains to check that $\{f(p_i, q_j)\}$ satisfies constraints 1 to 3. For the first constraint, we have $s_i \geq f(s, p_i) = \sum_{j=1}^n f(p_i, q_j)$ since s_i is the capacity of the edge (s, p_i) and conservation of flow at p_i implies the equality. The second constraint follows since c_{ij} is the capacity of the edge (p_i, q_j) . The third constraint is similar: the capacity of the edge (q_j, t) implies $b_j \geq f(q_j, t)$, and by conservation of flow, $f(q_j, t) = \sum_{i=1}^m f(p_i, q_j)$.

Efficiency: The graph G has $m + n + 2$ vertices and $m + n + mn$ edges. Constructing it takes $O(mn)$ time. The max-flow in G has value at most S since the cut $(\{s\}, V \setminus \{s\})$ has capacity S . Running Ford-Fulkerson on G takes $O(Smn)$ time, which dominates the time for constructing G . Thus the total running time is $O(Smn)$.

Problem 4: Carpooling (30 points)

A group of k people $p_1, \dots, p_k$ are carpooling to work for n days and they need to choose drivers for each day. However, only a subset of the people go to work each day. Also, they would like to divide up the driving duties fairly. In particular, no person should drive more than the expected number of times they would drive if the driver was chosen randomly each day (rounded up).

Example 0.1. *Let's say there are 3 people p_1, p_2, p_3 carpooling for 4 days and the people carpooling on each day are as follows:*

1. *On day 1, all 3 people are carpooling.*
2. *On day 2, p_1 and p_3 are carpooling.*
3. *On day 3, p_2 and p_3 are carpooling.*
4. *On day 4, p_2 and p_3 are carpooling again.*

In this case, if the driver was chosen randomly each day, the expected number of times p_1 drives is $\frac{1}{3} + \frac{1}{2} = \frac{5}{6}$. The expected number of times p_2 drives is $\frac{1}{3} + 2 \cdot \frac{1}{2} = \frac{4}{3}$. The expected number of times p_3 drives is $\frac{1}{3} + 3 \cdot \frac{1}{2} = \frac{11}{6}$. Thus, we can have p_1 drive at most 1 time and we can have p_2 and p_3 drive at most 2 times. One way to do this is to have p_1 drive on day 2, have p_2 drive on day 3, and have p_3 drive on days 1 and 4

We can state this problem more precisely as follows. We are given subsets the subsets of people $\{S_j\}$ who are carpooling each day and we want to choose drivers $\{p_{i_j}\}$ such that

1. $\forall j \in [n](i_j \in S_j)$ (the driver for each day is one of the people carpooling for that day)
2. $\forall i \in [k] \left(|\{j : i_j = i\}| \leq \left\lceil \sum_{j \in [n]: i \in S_j} \frac{1}{|S_j|} \right\rceil \right)$ (no person drives more than the expected number of times they should drive rounded up)

Show that such a driving schedule always exists, give an algorithm to find such a driving schedule, prove that your algorithm is correct, and analyze its runtime. The runtime of your algorithm should be polynomial in k and n .

Hint is available.

Solution

Construct the following network G :

- Create a source vertex s and a sink vertex t . For each day $j \in [n]$, create a vertex d_j and an edge (s, d_j) with capacity 1. For each driver $i \in [k]$, create a vertex p_i and an edge (p_i, t) with capacity $\left\lceil \sum_{j \in [n]: i \in S_j} \frac{1}{|S_j|} \right\rceil$.
- For each $j \in [n]$ and for each driver i who goes to work on day j , add an edge (d_j, p_i) . Set the capacity of the edge to be 1.

Then run Ford-Fulkerson on G . Let f be the resulting max-flow. We pick driver i on day j iff $f(d_j, p_i) = 1$.

Correctness: First we show that the max-flow value of G is n . It suffices to construct a flow with value n since the cut $(\{s\}, V \setminus \{s\})$ has capacity n . For $j \in [n]$, $i \in [k]$, let $f(s, d_j) = 1$, $f(d_j, p_i) = \frac{1}{|S_j|}$ for $i \in S_j$, and $f(p_i, t) = \sum_{j \in [n]: i \in S_j} \frac{1}{|S_j|}$. By construction, conservation of flow is satisfied and the value of this flow is $\sum_{j=1}^n f(s, d_j) = n$. It remains to check that the flow on each edge is at most the capacity. Indeed, $f(d_j, p_i) = \frac{1}{|S_j|} \leq 1$ and $f(p_i, t) \leq \lceil \sum_{j \in [n]: i \in S_j} \frac{1}{|S_j|} \rceil$, as desired.

Now, since the capacities are integers, Ford-Fulkerson will find an integral max-flow. Since the edges (d_j, p_i) have capacity 1, the flow on (d_j, p_i) is either 0 or 1. Since the capacity of the edge (s, d_j) is 1, by conservation of flow, for each j there is exactly one i such that $f(d_j, p_i) = 1$ and hence for each day we pick exactly one driver. By construction of G , driver i works on day j only if $i \in S_j$, so the first constraint is satisfied. And again, by conservation of flow, each driver i can be picked at most $\lceil \sum_{j \in [n]: i \in S_j} \frac{1}{|S_j|} \rceil$ many times, so the second constraint is also satisfied.

Efficiency: The graph G has $k + n + 2$ vertices and at most $n + k + nk$ edges. It takes $O(kn)$ time to construct. Since the max-flow value of G is precisely n , Ford-Fulkerson takes $O(kn^2)$ time, which dominates the time for constructing G . Thus, the total running time is $O(kn^2)$.

Hints:

2. What happens if your path from s to t goes back and forth across the minimum cut? Alternatively, can you construct an example where many possible paths from s to t are blocked by taking a single path?
3. You should reduce this problem to max flow. To prove that your algorithm is correct, you should do two things:
 1. Show that if there is a sales plan with total sales T then there is a flow with value T .
 2. Show that given an integer-valued flow with value T , you can extract a sales plan with total sales T .

Note that you can and should use Ford-Fulkerson as a black box so you do not need to justify why Ford-Fulkerson works.

4. Express this problem as a max flow problem with vertices corresponding to the people and vertices corresponding to the days. Can you construct a maximum flow (not necessarily an integer-valued flow) for this problem without running Ford-Fulkerson? Given this, what happens if you run Ford-Fulkerson on this problem?

For proving that your algorithm is correct, you can use Ford-Fulkerson as a black box so you just need to prove that you can extract a valid schedule from the flow given by Ford-Fulkerson.